

An Empirical Study on Focal Methods in Deep-Learning-Based Approaches for Assertion Generation

YIBO HE, Peking University, China

JIAMING HUANG, Peking University, China

HAO YU, Peking University, China

TAO XIE, Peking University, China

Unit testing is widely recognized as an essential aspect of the software development process. Generating high-quality assertions automatically is one of the most important and challenging problems in automatic unit test generation. To generate high-quality assertions, deep-learning-based approaches have been proposed in recent years. For state-of-the-art deep-learning-based approaches for assertion generation (DLAGs), the focal method (i.e., the main method under test) for a unit test case plays an important role of being a required part of the input to these approaches. To use DLAGs in practice, there are two main ways to provide a focal method for these approaches: (1) manually providing a developer-intended focal method or (2) identifying a likely focal method from the given test prefix (i.e., complete unit test code excluding assertions) with test-to-code traceability techniques. However, the state-of-the-art DLAGs are all evaluated on the ATLAS dataset, where the focal method for a test case is assumed as the last non-JUnit-API method invoked in the complete unit test code (i.e., code from both the test prefix and assertion portion). There exist two issues of the existing empirical evaluations of DLAGs, causing inaccurate assessment of DLAGs toward adoption in practice. First, it is unclear whether the last method call before assertions (LCBA) technique can accurately reflect developer-intended focal methods. Second, when applying DLAGs in practice, the assertion portion of a unit test is not available as a part of the input to DLAGs (actually being the output of DLAGs); thus, the assumption made by the ATLAS dataset does not hold in practical scenarios of applying DLAGs. To address the first issue, we conduct a study of seven test-to-code traceability techniques in the scenario of assertion generation. We find that the LCBA technique is not the best among the seven techniques and can accurately identify focal methods with only 43.38% precision and 38.42% recall; thus, the LCBA technique cannot accurately reflect developer-intended focal methods, raising a concern on using the ATLAS dataset for evaluation. To address the second issue along with the concern raised by the preceding finding, we apply **all seven test-to-code traceability techniques**, respectively, to identify focal methods automatically from **only test prefixes** and construct a new dataset named ATLAS+ by replacing the existing focal methods in the ATLAS dataset with the focal methods identified by the seven traceability techniques, respectively. On a test set from new ATLAS+, we evaluate four state-of-the-art DLAGs trained on a training set from the ATLAS dataset. We find that all of the four DLAGs achieve lower accuracy on a test set in ATLAS+ than the corresponding test set in the ATLAS dataset, indicating that DLAGs should be (re)evaluated with a test set in ATLAS+, which better reflects practical scenarios of providing focal methods than the ATLAS dataset. In addition, we evaluate state-of-the-art DLAGs trained on training sets in ATLAS+. We find that using training sets in ATLAS+ helps effectively improve the accuracy of the ATLAS approach and T5 approach over these approaches trained using the corresponding training set from the ATLAS dataset.

Authors' Contact Information: [Yibo He](#), Peking University, Key Lab of HCST (PKU), MOE; SCS, Beijing, China, yibohe@pku.edu.cn; [Jiaming Huang](#), Peking University, School of EECS, Beijing, China, 1900012931@pku.edu.cn; [Hao Yu](#), Peking University, School of Software and Microelectronics, Beijing, China, yh0315@pku.edu.cn; [Tao Xie](#), Peking University, Key Lab of HCST (PKU), MOE; SCS, Beijing, China, taoxie@pku.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

© 2024 Copyright held by the owner/author(s).

ACM 2994-970X/2024/7-ART78

<https://doi.org/10.1145/3660785>

CCS Concepts: • **Software and its engineering** → **Unit Testing**.

Additional Key Words and Phrases: Unit Testing, Assertion Generation, Traceability

ACM Reference Format:

Yibo He, Jiaming Huang, Hao Yu, and Tao Xie. 2024. An Empirical Study on Focal Methods in Deep-Learning-Based Approaches for Assertion Generation. *Proc. ACM Softw. Eng.* 1, FSE, Article 78 (July 2024), 22 pages. <https://doi.org/10.1145/3660785>

1 INTRODUCTION

Unit testing is widely recognized as an essential aspect of the software development process, helping to detect and diagnose bugs to ensure the high quality of software. However, writing effective unit test cases manually can be arduous and time-consuming. A research survey [5] indicates that developers spend, on average, 15.8% of their time during the entire development process writing test cases manually.

A unit test case can be divided into two parts: the test prefix and the test oracle. The **test prefix** comprises a method call sequence designed to drive the object under test into a specific state. In the unit test case shown in Listing 1¹, the test prefix involves instantiating a hashTable, inserting a key into the hashTable, and then deleting the key from the hashTable. The **test oracle** is the expected behavior of the focal method (i.e., the main method under test, serving as the central functionality examined in the corresponding unit test case), aiding in determining whether the behavior aligns with the requirements. In Listing 1, the test oracle is the assertion to verify the number of keys in the hashTable. Based on the test case name `removeKey` in Listing 1, we know that the primary objective of this test case is to test the functionality of `deleteKeyFromHashTable` (the focal method).

Listing 1. An Example of A Unit Test Case

```
@Test
void removeKey() {
    //Test prefix
    HashMapCuckooHashing hashTable = new HashMapCuckooHashing(10);
    hashTable.insertKey2HashTable(3);
    int initialSize = hashTable.getNumberOfKeysInTable();
    //Focal method invocation
    hashTable.deleteKeyFromHashTable(3);
    //Test oracle (assertion)
    assertEquals(initialSize - 1, hashTable.getNumberOfKeysInTable());
}
```

To alleviate the manual effort involved in writing unit test cases, various tools [4, 7, 20, 24, 42] have been proposed to generate unit test cases automatically. These tools mainly expose bugs by either regression oracles, where the observed return values of methods executed on the existing version of the program under test are converted into assertions [42], or detecting crashes and undesirable exceptions, e.g., division by zero. However, existing studies [3, 30] show that the quality of generated assertions (a common form of test oracle) is often inadequate when facing functional-correctness-related bugs, aka logic bugs.

To complement the preceding test-generation tools by automatically generating high-quality assertions, deep-learning-based approaches for assertion generation (DLAGs) [6, 18, 36, 39, 43], whose workflow is shown in Figure 1, have been proposed recently, falling into three main categories. (1) A **text-generation approach** [18, 36, 39] constructs assertions token by token, using neural network language models. *Focal-tests* (i.e., test prefixes and corresponding focal methods) are

¹<https://github.com/TheAlgorithms/Java>

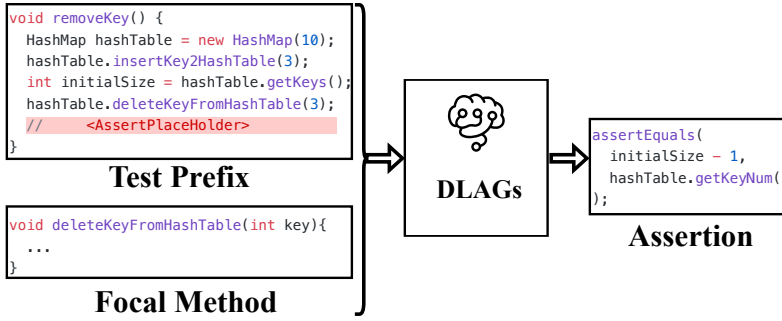


Fig. 1. Workflow of DLAGs

Table 1. Overview of DLAGs and their evaluation datasets

Tool/Study	Category	Venue	Eval
ATLAS [39]	Text generation	ICSE'20	ATLAS
T5 [18]	Text generation	ICSE'21	ATLAS
BART [36]	Text generation	AST'22	ATLAS
IR [43]	Information retrieval	ICSE'22	ATLAS
TOGA [6]	Classification	ICSE'22	ATLAS*

one part of imperative inputs for neural network language models to generate assertions. (2) An **information-retrieval (IR) approach** [43] deduces assertions by retrieving the assertion that best aligns with the given *focal-test* from the training set. The *focal-test* is considered as a comprehensive unit to calculate the similarity between the given *focal-test* and *focal-tests* in the training set, facilitating the retrieval of assertions. (3) A **classification approach** [6] generates a set of candidate assertions and classifies each candidate assertion as correct or incorrect. *Focal-tests* serve as inputs for generating and classifying a set of candidate assertions.

To use DLAGs in practice, there are two main ways to provide a focal method for these approaches. First, developers can manually provide the developer-intended focal method. Before writing a unit test case, developers typically already have a focal method in mind. Second, to alleviate the burden of manual identification, developers can use test-to-code traceability techniques as a preprocessing step to automatically identify a likely focal method from the given test prefix. However, as shown in Table 1, the assessment of the capability to accurately generate assertions by state-of-the-art approaches is conducted on the ATLAS dataset [39] or its variants (e.g., ATLAS* in TOGA [6]), where the focal method for a test case is assumed to be the last non-JUnit-API method invoked in the entire unit test code (i.e., code from both the test prefix and assertion portion).

There are two issues with the existing empirical evaluations of DLAGs, leading to an inaccurate assessment of DLAGs' suitability for practical adoption. First, it is unclear whether the last method call before assertions (LCBA) technique can faithfully reflect developer-intended focal methods. In some instances, the last method call before assertions is not the focal method. Second, in practical applications of DLAGs, the assertion portion of a unit test case is not available as part of the input to DLAGs. The assertion portion of a unit test case is actually the output of DLAGs.

To evaluate DLAGs with practical focal methods, in this paper, we present the first comprehensive study on focal methods. Given the significant value of automatic unit test generation, the surging development of DLAGs, and the substantial impact of data quality on deep learning tasks [2, 11],

our study is both timely and valuable. To address the first issue, we conduct a study using seven test-to-code traceability techniques in the scenario of assertion generation on a public dataset named TCTracer [40] with manually identified focal methods. We find that the LCBA technique does not rank as the best among the seven techniques, achieving only 43.38% precision and 38.42% recall in accurately identifying focal methods; thus, the LCBA technique fails to adequately reflect developer-intended focal methods, raising a concern regarding the use of the ATLAS dataset for evaluation. To tackle the second issue and address the concern raised by the preceding finding, we apply all seven test-to-code traceability techniques to identify focal methods automatically from only test prefixes. We construct a new dataset named ATLAS+ by replacing the existing focal methods in the ATLAS dataset with the focal methods identified by the seven traceability techniques.

Utilizing our new ATLAS+ dataset (which includes updated focal methods), we evaluate (1) DLAGs trained on a training set in the ATLAS dataset and (2) DLAGs trained on training sets in new ATLAS+, and obtain two main findings. First, we find that all of the four DLAGs achieve lower accuracy on a test set from ATLAS+ compared to the corresponding test set from the ATLAS dataset, indicating that DLAGs should be (re)evaluated with a test set from ATLAS+, which better reflects practical scenarios in focal method provision than the ATLAS dataset. Second, we find that using a training set from ATLAS+ effectively enhances the accuracy of DLAGs proposed by the ATLAS authors [39] and T5 authors [18] over DLAGs trained using the corresponding training set from the ATLAS dataset. For the IR approach and classification approach (TOGA), using a training set from ATLAS+ does not enhance the capability of these approaches.

In summary, this paper makes the following main contributions:

- An empirical study on the effectiveness of existing DLAGs when evaluated with focal methods identified by various traceability techniques.
- Adaptation and evaluation of test-to-code traceability techniques in the context of assertion generation.
- A new dataset named ATLAS+, which more accurately represents practical scenarios of providing focal methods than the ATLAS dataset.
- Insights on efficiently and effectively training DLAGs, along with a detailed analysis of the impact of focal methods on DLAGs.

All our scripts, datasets, and detailed instructions to reproduce DLAGs in this study are made publicly available [1].

2 BACKGROUND

This section provides background information on assertion generation and test-to-code traceability techniques. The task of assertion generation is, given a *focal-test* (namely a test prefix and a focal method), to generate assertions that determine whether the behavior of the given focal method is correct. Existing DLAGs generate only a single assertion without generating multiple assertions for a unit test case.

2.1 Deep-Learning-Based Approaches for Assertion Generation (DLAGs)

With the rapid advancement of deep learning, multiple DLAGs [6, 18, 36, 39, 43] are proposed to use deep learning techniques to tackle the task of assertion generation. These DLAGs (whose overview is shown in Table 1) can be categorized into three main groups based on their high-level strategies.

Text-generation approach. A text-generation approach [18, 36, 39] predicts a stream of tokens as an assertion by neural network language models. Watson et al. [39] introduce the first DLAG, named ATLAS, which applies the Sequence to Sequence (Seq2Seq) model [32] to generate assertions

given a *focal-test*. Additionally, Watson et al. also build the first dataset for assertion generation by collecting the pairs of $\langle \text{focal-test}, \text{assertion} \rangle$ from open-source Java projects on GitHub. The ATLAS dataset is widely used in DLAGs [6, 18, 36, 39, 43]. Subsequent work [18, 36] improves ATLAS by leveraging effective pre-trained models such as Text-To-Text Transfer Transformer (T5) [17, 28] and BART Transformer [14], with claims of achieving higher accuracy than ATLAS.

Information-retrieval approach. An information-retrieval (IR) approach [43] deduces assertions by retrieving the assertion that best aligns with the given *focal-test* from the training set. Yu et al. [43] propose an IR approach for generating assertions by calculating the Jaccard similarity [33] between the given *focal-test* and *focal-tests* in the training set. This IR approach retrieves the assertion corresponding to the *focal-test* with the highest similarity in the training set as the expected assertion. Yu et al. also propose an automatic adaptation technique that modifies at most one token in the retrieved assertion toward the correct one. The technique’s modification strategy employs deep learning models to determine whether the retrieved assertion should be modified, which token should be modified, and what candidate token should be replaced with.

Classification approach. A classification approach [6] generates a set of candidate assertions and classifies each candidate assertion as correct or incorrect. TOGA [6], one of state-of-the-art DLAGs, is a classification approach for assertion generation. TOGA generates assertions that conform to TOGA’s assertion grammar, where the assertion type is limited to one of “assertEquals”, “assertTrue”, “assertFalse”, “assertNull”, and “assertNotNull”. TOGA first creates a set of candidate assertions based on the return value of the given focal method. Subsequently, TOGA employs an assertion oracle ranker to classify these candidate assertions as correct (labeled with 1) or incorrect (labeled with 0). TOGA selects assertions labeled with 1 as the expected assertions. TOGA is implemented with EvoSuite [7] to generate entire unit test cases with high-quality assertions.

2.2 Test-to-Code Traceability Techniques

Test-to-code traceability techniques aim to establish an association from test code to the corresponding focal method. Various test-to-code traceability techniques have been proposed and applied in various domains, such as test generation [35], test comprehension [13, 44], and test maintenance [31, 37]. We next discuss five main categories of test-to-code traceability techniques.

Last method call. Last method call techniques select the last method call in a unit test case as the focal method, because developers often place assertions in a unit test case immediately after the method that developers intend to test. The last method call before assertions (LCBA) is a straightforward technique to establish the traceability between the unit test case and the focal method, and LCBA has been evaluated as a baseline in multiple studies [26, 29, 40, 41]. Watson et al. [39] use the LCBA technique to identify focal methods when building the ATLAS dataset. In addition to LCBA, Ghafari et al. [9] propose a technique to identify focal methods by detecting the last mutator method call, which is the last invoked method changing the state of the object under test, as opposed to observer methods, in unit test cases.

Lexical analysis. Lexical analysis techniques identify focal methods based on the assumption that the vocabulary in a unit test case (in the form of test method) and the vocabulary in its focal method are similar, such as variable names, identifiers, comments, and method implementation. Previous studies [25, 29] have indicated that lexical analysis of method implementation for identifying focal methods achieves low accuracy and recall. Given the naming conventions for unit test cases, the name of a unit test case and the name of its focal method are regularly similar and contain less noise compared to the vocabulary in the complete unit test code. Lexical analysis of method names is widely used in previous work [25, 26, 29, 35, 40, 41] to identify focal methods, employing analysis techniques such as containment relationship, longest common subsequence, and edit distance.

Program analysis. Program analysis techniques learn the intent of test code in order to accurately identify focal classes or methods. Qusef et al. [26, 27] use data flow analysis to identify the classes that affect parameter states in assertions of unit test cases. The identified classes are viewed as the focal class (i.e., the main class under test). Rompaey et al. [29] identify the focal class by constructing a static call graph to collect classes that are most referenced in a unit test case. White et al. [40] collect dynamic execution traces and analyze runtime information to establish traceability between focal methods and test code.

Combined rules. To improve the accuracy of test-to-code traceability techniques for identifying focal methods, various individual techniques are combined to collectively determine the focal method within a given unit test case. Rompaey et al. [29] evaluate the effectiveness of combined techniques, including naming convention (a lexical analysis technique), LCBA (a last method call technique), static call graph (a program analysis technique), etc. White et al. [40] use a combined score to calculate the similarity between the name of a unit test case and the name of each method invoked by the unit test case, establishing the test-to-code traceability when the similarity surpasses the designated threshold. Tufano et al. [35] identify focal methods by setting two rules: name matching (i.e., the invoked method's name is equal to the unit test case's name) and the unique method call (i.e., only one non-JUnit-API method is invoked in the unit test case). Invoked methods that meet any of the two rules are marked as focal methods.

Co-evolution. Unlike the preceding techniques that analyze source code, co-evolution techniques [29, 31] identify focal methods by version history in code repositories. These co-evolution techniques are based on the assumption that unit test cases should evolve in tandem with the corresponding focal methods.

3 METHODOLOGY

To evaluate the effectiveness of DLAGs when using them in practice, we adapt test-to-code traceability techniques to the scenario of assertion generation through a static analysis algorithm as shown in Algorithm 1. We construct our new dataset named ATLAS+ by modifying focal methods in the ATLAS dataset using seven adapted techniques. ATLAS+ includes 11 test sets and 4 training sets. We evaluate (1) the existing DLAGs trained on the training set in the ATLAS dataset and (2) DLAGs trained on the training sets in ATLAS+.

Additionally, we notice the rapid development of large language models (e.g., ChatGPT [23]) during our study. For the sake of fairness, we do not include this type of work in our experiments, because all the code in the ATLAS dataset constitutes part of the training data for large language models.

3.1 Adaption of Test-to-Code Traceability Techniques to Assertion Generation Task

Considering the reproducibility and consistency of our experiments, we opt to modify focal methods in the ATLAS dataset rather than constructing new datasets through mining Java projects on GitHub. Under this premise, our analysis is constrained solely to the source code of unit test cases, indicating that we should discard techniques reliant on any other data sources. Moreover, the feasibility of employing dynamic analysis techniques is precluded due to the impracticality and substantial time overhead associated with executing all unit test cases within the ATLAS dataset.

We provide detailed reasons why the unselected test-to-code traceability techniques are unsuitable for our study. The last mutator call technique requires analyzing the entire project repository rather than only unit test code. Lexical analysis of method implementations and static call graph analysis are discarded due to previous findings [25, 29] indicating the two techniques' low accuracy and recall. Program analysis techniques are also excluded because the existing implementations of these techniques [26, 27] are applicable for only class-level traceability. Co-evolution techniques

Algorithm 1 Adaption of Test-to-code Traceability Techniques

Require: test: a unit test case with implementation
Require: technique: a test-to-code traceability technique
Ensure: focal: the focal method identified by this technique

```

1: procedure GETFOCALMETHOD(test, technique)
2:   AST  $\leftarrow$  Parse(test)
3:   nodeList  $\leftarrow$  DepthFirstTraverse(AST)
4:   maxScore  $\leftarrow$  0, focal  $\leftarrow$  ""
5:   for node  $\in$  nodeList do
6:     if node.type == MethodInvocation then
7:       nodeScore  $\leftarrow$  GetFocalScore(test, node, technique)
8:       if nodeScore  $\geq$  maxScore then
9:         maxScore  $\leftarrow$  nodeScore
10:        focal  $\leftarrow$  node
11:   return focal
12: procedure GETFOCALSCORE(test, node, technique)
13:   nodeScore  $\leftarrow$  case technique of
14:   | NC : test.name.deletePrefix("test").equals(node.name)
15:   | NCC : test.name.deletePrefix("test").contains(node.name)
16:   | LCSB :  $\frac{LCS(test.name, node.name).len()}{Max(node.name.len(), test.name.len())}$ 
17:   | LCSU :  $\frac{LCS(test.name, node.name).len()}{node.name.len()}$ 
18:   | ED :  $1 - \frac{EditDistance(test.name, node.name)}{Max(node.name.len(), test.name.len())}$ 
19:   | LCBA : last_call_before_assert(test).equals(node)
20:   | CS :  $\frac{\sum_{i=1}^n Prec_i * Score_i(test, node)}{\sum_{i=1}^n Prec_i}$ 
21:   return nodeScore

```

require the access to version history in code repositories. Besides, we discard all dynamic analysis techniques, such as Term Frequency–Inverse Document Frequency (TF-IDF) [8] used in previous studies [40, 41]. After filtering existing test-to-code traceability techniques introduced in Section 2.2, we select seven techniques appropriate for assertion generation scenarios: the last call before assertions (LCBA), naming convention (NC), naming convention-contain (NCC), longest common subsequence (*LCS_B*, *LCS_U*), edit distance (ED), and the combined-score (CS) technique.

The process of adapting test-to-code traceability techniques to the assertion generation task is shown in Algorithm 1. For each unit test case, we generate an abstract syntax tree (AST) using the Javalang [34] library, a static analysis tool for Java programs. Then we traverse the AST and collect method invocation nodes in the tree. For each technique, we select the invoked method with the highest score as the focal method identified by this technique. In cases where multiple invoked methods have the highest score, we select the last-invoked method as the focal method. Although one unit test case may correspond to multiple focal methods in practical scenarios, we believe that identifying a single focal method is reasonable because existing DLAGs require the focal method most closely associated with the single target assertion. If there are no invoked methods or if all invoked methods have a score of 0, an empty string is returned as the focal method.

We calculate a similarity score between each invoked method and its corresponding unit test case (namely test method), as shown in the *GetFocalScore* procedure in Algorithm 1. The score ranges from 0 to 1 for each adapted technique. Specifically, for NC, NCC, and LCBA, the score is

a binary value (0 or 1), and for LCS_B , LCS_U , ED, and CS, the score is a continuous value between 0 and 1. The greater the score value, the more similar the two methods are. For NC, the score is 1 if and only if the invoked method's name is equal to the unit test case's name (after the prefix "test" is removed if it exists in the unit test case's name). For NCC, the score is 1 if and only if the invoked method's name is a substring of the unit test case's name (after the prefix "test" is removed if it exists in the unit test case's name). For LCBA, the score is 1 if and only if the invoked method is the last call before assertions in the unit test case. For LCS_B and LCS_U , we first calculate the length of the longest common subsequence between the name of the invoked method and the name of the unit test case, and then normalize the length using the maximum length of the two names (for LCS_B) or the length of the invoked method's name (for LCS_U). For ED, the score is 1 minus the edit distance divided by the maximum length of the two names. The edit distance between two words is the minimal number of single-character edits, including insertions, deletions, or substitutions, required to transform one word into the other. To comprehensively consider each adapted technique, we set the combined score used in CS as the weighted average of the individual techniques' scores, and set the weights as the precision of corresponding techniques. As shown in the GetFocalScore procedure in Algorithm 1, $Prec_i$ is the precision of technique_{*i*} on the TCTracer dataset [40], and $Score_i$ is the similarity score between the given unit test case (ut) and invoked method (im) calculated by technique_{*i*}. Given a unit test case, for each invoked method, we first calculate similarity scores using the first six techniques, and then calculate the weighted average score of the six techniques when using the CS technique.

3.2 Construction of ATLAS+

3.2.1 Benchmarks (test sets) in ATLAS+. Consistent with the two main ways to use DLAGs in practice (as discussed in Section 1), we construct our benchmarks by replacing the last non-JUnit-API methods in the test set of the ATLAS dataset with invoked methods that are (1) as closely aligned with the developer-intended focal methods as possible or (2) identified by test-to-code traceability techniques from test prefixes rather than complete test code including assertion portion.

Benchmarks with likely developer-intended focal methods. We select NC to identify likely developer-intended focal methods because NC achieves the highest (100%) precision in the experiment detailed in Section 4.1. We keep only cases where NC successfully identifies invoked methods with a score of 1, and we finally obtain 432 unit test cases. We manually check whether the identified focal methods of these cases are developer-intended focal methods, involving three participants with an average of four years of Java development experience, and we verify that all the identified focal methods of the 432 test cases are highly likely developer-intended focal methods. We replace the focal methods in the test set of the ATLAS dataset with the verified methods. The corresponding entries in the ATLAS dataset are retained as a control group.

Benchmarks with focal methods identified by traceability techniques from test prefixes. We replace the last non-JUnit-API methods in the test set of the ATLAS dataset using seven test-to-code traceability techniques selected as discussed in Section 3.1, resulting in seven benchmarks. In some cases, a traceability technique fails to identify an invoked method as the focal method, when either there are no invoked methods or all the invoked methods receive a score of 0, thereby returning an empty string as the focal method. Table 2 shows the number of failed cases for each technique. A case is viewed as failed for a technique when there is at least one invoked focal method in the test case and all the invoked methods receive a score of 0 determined by the technique. Besides the failed cases, there are 658 cases in each benchmark where there are no invoked methods. We set up two control groups: one is the original test set from the ATLAS dataset, and the other, named ATLAS-, is the test set from the ATLAS dataset resulted from removing the implementations of focal methods (to evaluate the impact of focal method implementations).

Table 2. Number of Failed Cases for Each Technique in Benchmark Construction

Technique	NC	NCC	LCS_B	LCS_U	ED	LCBA	CS
Failed Num.	16228	13789	38	38	123	0	0

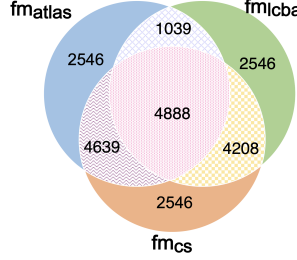


Fig. 2. Complementarity of Focal Method Identification in Benchmark Construction

3.2.2 Training sets in ATLAS+. To evaluate the effectiveness of DLAGs trained with updated focal methods, we also update focal methods in the training set of the ATLAS dataset. In TS_{atlas} , we retain only the signatures of focal methods in the ATLAS dataset and eliminate the implementations of focal methods. In TS_{null} , we configure focal methods to an empty string, signifying that only test prefixes serve as the input for DLAGs. In TS_{lcba} , we apply the LCBA technique, where the information in the assertion portion is not used. We also apply the CS technique, which performs the best in terms of recall and F1 score for identifying focal methods (evaluation of test-to-code traceability techniques is shown in Section 4.1), and we name this training set as $TS_{combined}$.

Our new dataset ATLAS+, including 4 training sets and 11 test sets, differs from the ATLAS dataset in three aspects. First, the ways to identify focal methods are different as discussed earlier. Second, we remove all unit test cases that cannot be parsed correctly by Javalang². To avoid the impact of potential bugs in Javalang, we conduct a manual investigation of a randomly sampled subset comprising 100 unit test cases that fail to be parsed. We find that all sampled unit test cases contain syntax errors, such as mismatched parentheses, ‘try’ blocks without corresponding ‘catch’ blocks. The removed cases from the ATLAS dataset are publicly available in our repository [1]. Third, we retain the names and argument types of focal methods, and remove the implementations of focal methods. We evaluate the effect of removing the implementations of focal methods by comparing results from models trained on the ATLAS training set and TS_{atlas} , and we discuss the evaluation results in Section 4.3. Additionally, removing the implementations of focal methods can bring two additional benefits. (1) Removing the implementations of focal methods can simplify our steps of dataset construction. Tracing from the entries of the ATLAS dataset to the corresponding implementations in repositories is extremely laborious and unrealistic. (2) Assertions (test oracles) should not rely on potentially buggy implementations of focal methods, as noted by some DLAGs, e.g., TOGA [6], which have refrained from using the implementations of focal methods.

We provide the complementarity analysis of focal method identification techniques that are used in the construction of ATLAS+ as shown in Figure 2. Statistics show that 65.78% of the focal methods identified by ATLAS in the test set differ from those identified by LCBA, and 44.99% of the focal methods identified by ATLAS in the test set differ from those identified by CS.

²<https://github.com/c2nes/javalang>

Table 3. Overview of the ATLAS+ Dataset's Composition

	id	Source	#items	tm	fm_sig ¹	fm_impl ²	assert_in ³
Test sets	1	ATLAS	17318	✓	✓	✓	✓
	2	ATLAS-	17318	✓	✓	✗	✓
	3	LCBA	17318	✓	✓	✗	✗
	4	NC	17318	✓	✓	✗	✗
	5	NCC	17318	✓	✓	✗	✗
	6	LCS_B	17318	✓	✓	✗	✗
	7	LCS_U	17318	✓	✓	✗	✗
	8	ED	17318	✓	✓	✗	✗
	9	CS	17318	✓	✓	✗	✗
	10	ATLAS	432	✓	✓	✓	✓
	11	NC	432	✓	✓	✓	✗
Training sets	1	ATLAS	138516	✓	✓	✗	✓
	2	NULL	138516	✓	✗	✗	✗
	3	LCBA	138516	✓	✓	✗	✗
	4	CS	138516	✓	✓	✗	✓

¹Including signatures of focal methods or not.²Including implementations of focal methods or not.³Whether the information in expected assertions is included when identifying focal methods.

3.3 Study Setup

Selection of DLAGs. There are five DLAGs (as shown in Table 1) that align with our definition of the assertion generation task in Section 2. The BART work [36] is not included in our study, because its source code and trained models are not available.

Setup. We conduct our experiments on a server equipped with two NVIDIA GeForce RTX 3090 GPUs (24GB GPU Memory for each card), two AMD EPYC™ 7H12, and 512GB of RAM. For ATLAS, we use a TensorFlow docker container [22] provided by NVIDIA to run the TensorFlow code on GPUs. For IR, we run its public code directly on our server. For TOGA, we use its provided docker container [19]. Note that, for T5, we do not run it on our server but use a 2×2 TPU topology from Google Colab. All other experimental configurations remain consistent with their original setups.

Metrics. We use three metrics in our experiments: accuracy, BLEU, and generation time. Accuracy, also known as perfect prediction percentage, is one of the primary metrics assessing the effectiveness of DLAGs [28, 39, 43]. An assertion is deemed perfect only when it aligns precisely with the ground truth, and accuracy is the ratio of perfectly predicted assertions to all predicted assertions. If we use beam search in DLAGs, we set the K value to 1. BLEU (Bilingual Evaluation Understudy) is a well-known metric to measure machine translation effectiveness. The fundamental concept behind BLEU is that the better a machine translation approximates a professional human translation, the higher its quality. Following the previous work [39, 43], we calculate the Multi-BLEU score using the Perl script provided by Moses [21]. The script computes the modified n-gram precision (for n=1,2,3,4) of a candidate sequence (i.e., generated assertions) concerning the reference sequence (i.e., expected assertions). Subsequently, the script assesses the average modified n-gram precision, taking into account penalties for excessively brief sequences. Generation time is the total time for inference on the test set, used to measure the performance of DLAGs. For ATLAS and IR, we collect generation time by using the “time” command in Linux. For T5 and TOGA, we collect generation time via Python’s time library.

Table 4. Results of Test-to-code Traceability Techniques Adapted to the Assertion Generation Task

Technique	Precision	Recall	F1
NC	100.00	1.48	2.91
NCC	37.50	1.97	3.74
LCS_B	63.87	49.26	55.62
LCS_U	76.79	55.67	64.54
ED	69.35	51.72	59.25
LCBA	43.48	38.42	40.79
CS	69.98	64.53	67.15

TOGA is special compared to other approaches because it can generate only assertions that conform to TOGA's grammar. To evaluate the effectiveness of TOGA, we use overall accuracy and in-template accuracy. The overall accuracy is the accuracy discussed earlier, and we regard cases that mismatch TOGA's grammar as failed cases. The in-template accuracy is the ratio of perfectly predicted assertions to assertions that conform to TOGA's grammar.

4 RESEARCH QUESTIONS AND FINDINGS

In this section, we detail the experiment results, analysis, and findings of our study on focal methods in DLAGs.

4.1 RQ1: Effectiveness of Adapted Traceability Techniques

In this section, we answer RQ1 (*how effective are adapted test-to-code traceability techniques in identifying focal methods in the scenario of assertion generation?*) by measuring precision, recall, and F1 score achieved by each adapted test-to-code traceability technique on the TCTracer [40] dataset, where unit test cases are labeled with ground-truth focal methods.

Different from the ATLAS dataset, some of the unit test cases in the TCTracer dataset contain multiple assertions. To be consistent with the assertion generation task defined in Section 2, we split unit test cases that have multiple assertions into ones each with a single assertion. Specifically, for each assertion in a unit test case, we combine the non-assertion statements before the assertion with the assertion as a new split unit test case. Through this splitting process, we obtain 573 unit test cases from TCTracer, each containing no more than one assertion. We evaluate the adapted test-to-code traceability techniques discussed in Section 3.1 on the split TCTracer dataset. The results of this experiment are shown in Table 4.

Results and analysis of the NC and CS techniques. The NC technique achieves the highest (100%) precision among all the adapted techniques, indicating that if an invoked method has the same name as its unit test case (after the prefix “test” is removed if it exists), e.g., `Push()` and `testPush()`, this invoked method has a high probability of being the focal method for this unit test case. Unfortunately, the NC technique also achieves the lowest recall, indicating that in rare instances, unit test cases invoke a method with the same name. Overall, the CS technique that takes into account all six techniques performs the best in our experiment with the highest recall, the highest F1 score, and the third-highest precision.

Results and analysis of other techniques. We find that the precision of the NCC technique drops substantially compared with that of the NC technique, and the recall of the NCC technique shows little improvement. A common case is when candidate methods have short names (comprising only a few characters) that are substrings of test method names, the NCC technique tends to yield numerous false positives. Changing the requirement of naming convention from equality to

containing leads to a large proportion of false positives. The LCS_B, LCS_U, and ED techniques are also well performing in identifying focal methods. Unit test methods and their corresponding focal methods have a high probability of sharing common vocabulary in their method names. In approximately only 40% of cases, the last method call before assertions in a test case coincides with the focal method. LCBA is not the best in terms of precision, recall, and F1 score.

Finding 1: The NC technique adapted to the assertion generation task achieves the highest (100%) precision, although its recall is low. The CS technique adapted to the task performs the best to identify focal methods in our experiment with the highest F1 score (69.98% precision, 64.53% recall, and 67.15% F1 score).

We admit that the effectiveness of adapted test-to-code traceability techniques can be evaluated more accurately on a large-scale dataset than solely on the TCTracer dataset; however, we carry out the experiment on only the TCTracer dataset for two main reasons. First, focal method identification should be evaluated with ground-truth focal methods; however, we have tried our best to find datasets with ground-truth focal methods, and we find TCTracer as the only available dataset with ground-truth focal methods. Second, manually building a large-scale dataset with ground-truth focal methods is highly costly and, more importantly, infeasible without direct access to the engineers who write the test cases.

4.2 RQ2: Effectiveness of Existing Models on Test Sets in ATLAS+

In this section, we answer RQ2 (*how effective are existing deep-learning-based assertion generation models³ evaluated on the benchmarks where focal methods are close to practice?*). As discussed in Section 1, there are two main ways to provide a focal method for DLAGs in practice: (1) manually providing a developer-intended focal method or (2) identifying a likely focal method from the given test prefix with test-to-code traceability techniques. Specifically, to evaluate existing DLAGs with practical focal methods, we use our benchmarks with focal methods (1) successfully identified by the NC technique and manually checked by three participants to ensure that focal methods are as close to developer-intended as possible or (2) identified by all evaluated traceability techniques from test prefixes rather than complete test code including the unknown assertion portion.

For T5, IR, and TOGA, we successfully execute their code and available models. However, in the case of ATLAS, we find that two required documents (train_options.json and checkpoint) are missing in the existing ATLAS model [38], resulting in our inability to execute the ATLAS model. To overcome this obstacle, we reproduce ATLAS by training new models on the ATLAS dataset instead of using the available model.

4.2.1 Focal methods identified by adapted traceability techniques from test prefixes. We compare the results of existing deep-learning-based assertion generation models on the test set of the ATLAS dataset and on our benchmarks with focal methods identified by adapted test-to-code traceability techniques from test prefixes.

Delta. Figure 3 shows the accuracy results of our experiment. We observe that all of the DLAGs under study achieve lower accuracy on our benchmarks than on the test set of the ATLAS dataset. We show the changes in accuracy between the test set of the ATLAS dataset and the average of our eight benchmarks: ATLAS from 21.28% to 16.18%, T5 from 48.47% to 28.88%, IR from 43.33% to 37.21%, and TOGA from 36.15% to 33.20%. The accuracy of T5 decreases by 40.42%, which is the most of the four approaches under study. The accuracy of TOGA decreases by only 8.16%,

³Publicly available or reproduced models of DLAGs.

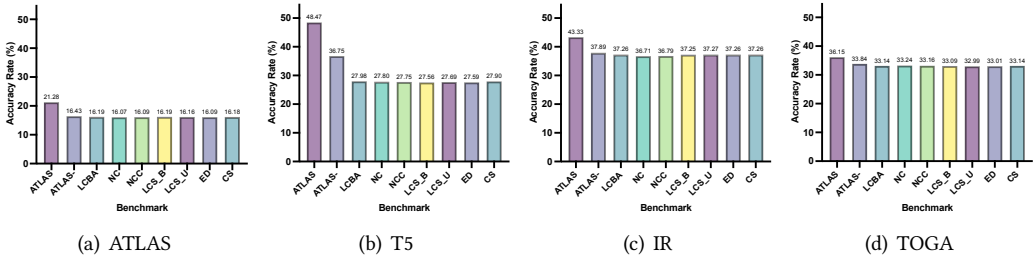


Fig. 3. Accuracy Results of Existing Deep-Learning-Based Models for Assertion Generation on Benchmarks with Focal Methods Identified by Adapted Traceability Techniques from Test Prefixes (Total=17318)

Table 5. BLEU Results of Existing Deep-Learning-Based Models for Assertion Generation

Benchmark	ATLAS	ATLAS-	LCBA	NC	NCC	LCS_B	LCS_U	ED	CS
ATLAS	54.67	49.69	49.31	48.97	49.02	49.28	49.30	49.24	49.28
T5	77.25	66.55	55.25	56.08	56.14	55.97	55.81	55.91	55.30
IR	77.76	70.88	70.08	69.57	69.63	70.07	70.07	70.08	70.09

making it the least affected of the four approaches under study. As shown in Table 5, the BLEU score results are almost consistent with the accuracy results for ATLAS, T5, and IR. In most cases, for each approach on the same benchmark, higher accuracy correlates with a greater BLEU score. We observe that the original BLEU scores of T5 are notably low, e.g., 3.85 on the test set of the ATLAS dataset, and we find out the reason to be that all the tokens in assertions generated by T5 are lowercase and T5 uses a different tokenizer model. Before calculating the BLEU scores of T5, we pre-process the assertions generated by T5 to ensure that all tokens are transformed into the same format as tokens in the expected assertions in benchmarks. Regarding TOGA, a substantial quantity of candidate assertions labeled with 0 are generated, making the BLEU score meaningless.

Ranking. We notice that the accuracy ranking of DLAGs changes. When evaluated on the test set of the ATLAS dataset, T5 achieves the highest accuracy followed by IR. However, when evaluated on our benchmarks, IR achieves the highest accuracy followed by TOGA. Similarly, when evaluated using the BLEU score metric, IR and T5 are indistinguishable on the test set of the ATLAS dataset. However, IR achieves better BLEU than T5 on benchmarks in ATLAS+. The update on focal methods in benchmarks has the greatest impact on the accuracy of the T5 model. ATLAS that applies the Seq2Seq model achieves the lowest accuracy on both the test set of the ATLAS dataset and our benchmarks.

Reproducibility. We can reproduce the claimed accuracy results [6, 28, 39, 43] for all the four DLAGs under study. For ATLAS, T5, and IR, the accuracy of the three approaches slightly improves when tested on our cleaned test set derived from the ATLAS dataset than their claimed accuracy. The little difference is likely to be caused by removing unit test cases with syntax errors in the test set of the ATLAS dataset. For TOGA, we find a decrease in accuracy from the claimed accuracy to the accuracy that we reproduce (overall accuracy from 69% to 36%, and in-template accuracy from 96% to 93%). The decrease in overall accuracy is mainly due to that our calculation as mentioned in Section 3.3 is different from the calculation in TOGA's implementation [6] given the fairness of comparison with other DLAGs. TOGA's implementation considers only five assertion types, but we do not have this restriction. The decrease in in-template accuracy is mainly due to a neglected

Table 6. Results of Existing Deep-Learning-Based Models for Assertion Generation on Benchmarks with Likely Developer-intended Focal Methods (Total=432)

Test Set \ Approach	Accuracy				BLEU		
	ATLAS	T5	IR	TOGA	ATLAS	T5	IR
ATLAS	19.91	62.73	40.05	25.93	52.05	84.68	74.72
NC	16.90	32.17	35.42	22.69	49.88	60.15	70.57

bug in construction of TOGA's benchmark (ATLAS*). This bug as well as our solution for it has been confirmed by TOGA's authors.

Finding 2: ATLAS, T5, IR, and TOGA all exhibit lower accuracy on our benchmarks with focal methods identified by adapted test-to-code traceability techniques from test prefixes compared to the original test set of the ATLAS dataset. IR achieves the highest accuracy (average: 37.21%) on our benchmarks. TOGA's behavior is the most stable (decline: 8.16%).

4.2.2 Likely developer-intended focal methods. To evaluate models for assertion generation when facing developer-intended focal methods, we collect cases where NC, achieving 100% precision in our experiment, can successfully identify focal methods from test prefixes, obtaining 432 cases. Three participants with an average of four years of Java development experience manually check whether the identified focal methods of these cases are the most likely developer-intended focal methods. We also keep the corresponding entries in the ATLAS dataset as a control group. Accuracy and BLEU results are recorded for analysis.

Delta. The accuracy and BLEU results are shown in Table 6. The accuracy of ATLAS, T5, IR, and TOGA is all lower on the NC benchmark with likely developer-intended focal methods compared to the control group. Among these four DLAGs, the accuracy of T5 exhibits the most drop (from 62.73% to 32.17%). The BLEU scores for ATLAS, T5, and IR also decrease.

Ranking. T5 achieves the highest accuracy in the control group; however, IR achieves the highest on the NC benchmark where focal methods are likely developer-intended. ATLAS achieves the lowest accuracy on both of the small-scale benchmarks. The pattern of ranking changes is almost the same as that in the experiment in Section 4.2.1.

Finding 3: ATLAS, T5, IR, and TOGA all exhibit lower accuracy on the small-scale NC benchmark with likely developer-intended focal methods identified by the NC technique compared to the control group. IR achieves the highest accuracy (35.42%) on the NC benchmark.

The only difference between the test set of the ATLAS dataset and our benchmarks is that focal methods are updated in our benchmarks, and focal methods are changed in two main ways. One is the change of techniques for the identification of focal methods, and the other is the removal of focal method implementations in our benchmarks. Besides, cases with syntax errors are also removed. By analyzing the results of T5 in Figure 3(b), we know that the accuracy of T5 decreases on the ATLAS- benchmark compared to the accuracy on the test set of the ATLAS dataset, and the accuracy on other benchmarks in ATLAS+ further decreases compared to the accuracy on the ATLAS- benchmark, indicating that both changes can cause a decrease in the accuracy of T5. However, for ATLAS, IR, and TOGA, we know that only the accuracy of these approaches drops when facing likely practical focal methods, and we cannot determine which part of the change

Table 7. Results of ATLAS, T5, IR, and TOGA on training sets in ATLAS+

Approach	Dataset	Accuracy	BLEU	Count
ATLAS	ATLAS	21.28	54.67	3,685
	TS_{atlas}	24.36	56.77	4,220
	TS_{null}	19.54	50.25	3,385
	TS_{lcba}	20.00	52.87	3,464
	$TS_{combined}$	29.03	59.61	5,028
T5	ATLAS	49.37	75.35	8,549
	TS_{atlas}	57.40	80.44	9,940
	TS_{null}	53.06	77.14	9,188
	TS_{lcba}	52.34	77.08	9,065
	$TS_{combined}$	51.66	75.49	8,946
IR	ATLAS	43.33	77.76	7,504
	TS_{atlas}	43.60	76.83	7,551
	TS_{null}	42.49	75.45	7,360
	TS_{lcba}	42.42	75.50	7,347
	$TS_{combined}$	43.25	76.51	7,491
Approach	Dataset	Accuracy Overall	Accuracy In-Template	Count
TOGA	ATLAS	31.06	80.25	5,379
	TS_{atlas}	31.31	80.90	5,423
	TS_{null}	32.08	82.88	5,556
	TS_{lcba}	31.99	82.65	5,541
	$TS_{combined}$	31.67	81.82	5,485

is the exact cause of the decrease in the accuracy. To address this question, we conduct further comparative experiments where models are trained on the dataset with ATLAS' focal methods but without their implementations (TS_{atlas}). The results of these experiments are shown and discussed in Section 4.3.

4.3 RQ3: Effectiveness of DLAGs Trained on Training Sets in ATLAS+

In this section, we address RQ3 (*how effective are DLAGs trained on training sets in ATLAS+?*). In Section 4.2, we find that DLAGs perform the best on the benchmark where focal methods are identified using the same technique as the training set. For each training set, we evaluate the DLAGs using the corresponding test set, where focal methods are identified by the same technique as the training set, to ensure the models' effectiveness. All training configurations remain the same as the experiments in their initial work for ATLAS, IR, and TOGA. We finetune the public code-pretrained T5 model [18] on the single task of assertion generation for each training set in ATLAS+. This approach is different from their initial work, where the T5 model is fine-tuned on a mixture of tasks. All models to answer RQ3 are trained by ourselves to ensure that the models are trained in the same training software/hardware environments for fairness, and this setting is different from RQ2, where we use existing fine-tuned models provided by the research community.

Comparison between ATLAS and ATLAS+. By comparing the results between ATLAS and ATLAS+, we find that the accuracy and BLEU results of ATLAS and T5 can be improved by training

with updated focal methods in ATLAS+. However, for IR and TOGA, there is almost no difference in the results. This comparison allows us to address the question mentioned at the end of RQ2: the decline in the accuracy of both IR and TOGA primarily results from the elimination of the focal method implementations, as the identification of the focal method does not affect these two approaches. For ATLAS and T5, removing the focal method implementations can bring positive effects. The only difference between the ATLAS dataset and TS_{atlas} is the inclusion of the focal method implementations. By comparing the results of the ATLAS and T5 approaches between the ATLAS dataset and TS_{atlas} , we find that the accuracy and BLEU results can be improved for both approaches. This finding indicates the superiority of training sets in ATLAS+, and we recommend that developers remove the focal method implementations when training ATLAS and T5. How focal methods impact or do not impact DLAGs is discussed in Section 5.2.

Finding 4: ATLAS and T5 are sensitive to focal methods, whereas IR and TOGA are not. Updating focal methods and removing their implementations from the training set can improve the accuracy and BLEU scores of ATLAS and T5.

Comparison between training sets in ATLAS+. Table 7 shows the accuracy and BLEU results of ATLAS, T5, IR, and TOGA that are trained on our training sets in ATLAS+. We observe that the accuracy and BLEU results of ATLAS and T5 differ substantially, indicating that ATLAS and T5 are sensitive to focal methods. For ATLAS, training with focal methods identified by the CS technique can improve the accuracy and BLEU for assertion generation of Seq2Seq models used in ATLAS (from 19.54% to 29.03%). However, for T5, the accuracy and BLEU results cannot be improved, and may even be slightly degraded, by training with focal methods identified by the CS technique compared to other training sets in ATLAS+. A plausible explanation is that focal methods identified by the CS technique can change the convergence point of the model, leading to a change in the capability of assertion generation. This change can have both beneficial and detrimental effects. Moreover, ATLAS on TS_{atlas} achieves 21.80% relative improvement (from 20.00% to 24.36%) over the model on TS_{lcb} , and T5 on TS_{atlas} achieves 9.67% relative improvement (from 52.34% to 57.40%) over the model on TS_{lcb} . Considering that the main difference between TS_{atlas} and TS_{lcb} is that the information in the assertion portion is blocked when identifying focal methods in TS_{lcb} but not in TS_{atlas} , one plausible explanation is that focal methods in expected assertions can be hints for assertion generation, e.g., hints of the observer methods that are invoked in assertions, and sequence models can learn from these hints.

Another observation is that the accuracy results of IR and TOGA are almost not changed on the four training sets in ATLAS+, and we provide the analysis to explain this observation after further investigating the implementations of IR and TOGA. For IR, predicted assertions are retrieved by calculating the Jaccard similarity between the given *focal-test* and *focal-tests* in the training set. When a retrieved assertion matches the expected assertion, the similarity of *focal-tests* and the similarity of unit test cases without focal methods are both high, and the premise is that the focal method identification approaches in the training set and test set are consistent. For TOGA, we find that focal methods are almost not used in TOGA's implementation. Focal methods are designed to be used for creating a set of candidate assertions based on the return value of focal methods. However, in TOGA's actual implementation [19], TOGA generates candidate assertions mainly by the type of expected assertions rather than by the return value of focal methods as designed.

Ranking. T5 achieves the highest average accuracy and the highest average BLEU scores when trained on ATLAS+, followed by IR, TOGA, and ATLAS. ATLAS performs the best on $TS_{combined}$.

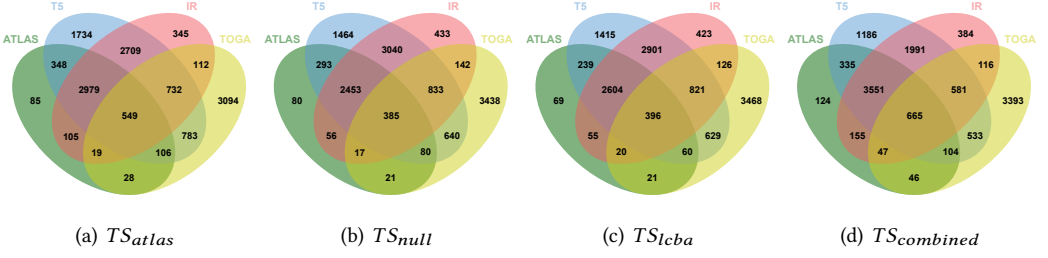


Fig. 4. Complementarity among ATLAS, T5, IR, and TOGA trained on ATLAS+

T5 performs the best on TS_{atlas} . Accuracy and BLEU results of IR and TOGA basically have no difference among training sets in ATLAS+.

Finding 5: ATLAS performs the best on $TS_{combined}$. T5 performs the best on TS_{atlas} , and T5 achieves the highest average accuracy and average BLEU results among the four DLAGs trained on ATLAS+.

Complementarity. Complementarity results in successful cases for the four DLAGs are shown in Figure 4. Only 4%-6% of IR successful cases are unique, i.e., can successfully generate expected assertions by only IR. Only 2%-3% of ATLAS successful cases are unique. However, this ratio can be around 15% for T5 and 60% for TOGA. Most of the assertions generated correctly by ATLAS and IR can also be generated correctly by T5 and TOGA. Although the accuracy of IR is high (1st in RQ2 and 2nd in RQ3), IR provides less unique value compared to T5 and TOGA. Combining the T5 and TOGA approaches together toward more accurate assertion generation can be a valuable future direction. Moreover, we observe that the overall proportions in Figures 4(a), 4(b), 4(c), and 4(d) are generally consistent, indicating that there are few effects of focal methods on the complementarity among ATLAS, T5, IR, and TOGA trained on the four training sets in ATLAS+.

Finding 6: ATLAS and IR provide less unique value compared to T5 and TOGA. Focal methods have few effects on the complementarity among ATLAS, T5, IR, and TOGA trained on the four training sets in ATLAS+.

Time cost. Table 8 shows the generation time results of ATLAS, T5, IR, and TOGA. We can immediately notice that the generation time on the ATLAS dataset is more than any other four training sets in ATLAS+ for all the four DLAGs. In particular, the ATLAS approach requires 29% less time, IR requires 42% less time, and TOGA requires 4% less time to generate assertions on TS_{null} (where focal methods are set to empty strings) compared to the original ATLAS dataset. In the construction of ATLAS+, we remove the implementations of focal methods. The results show that removing focal method implementations in a training set can substantially reduce time cost for DLAGs.

In addition, we compare the time cost of different approaches on the same dataset. We find that IR faces huge time cost on all five datasets, despite achieving high accuracy. For example, on TS_{null} , the time cost of IR is 142 times that of ATLAS, 216 times that of T5, and 687 times that of TOGA. It takes only a few minutes for ATLAS, T5, and TOGA to generate assertions for the entire test set; however, it takes more than 12 hours for IR.

Table 8. Generation Time (seconds) with Delta¹ of ATLAS, T5, IR, and TOGA

Training	ATLAS	TS_{atlas}	TS_{null}	TS_{lcb}	$TS_{combined}$
ATLAS	448(0%)	320(-28.57%)	319(-28.79%)	317(-29.24%)	306(-31.70%)
T5	262(0%)	215(-18.10%)	210(-19.91%)	194(-26.05%)	232(-11.38%)
IR	77,823(0%)	47,393(-39.10%)	45,323(-41.76%)	49,972(-35.79%)	48,971(-37.07%)
TOGA	69(0%)	64(-7.25%)	66(-4.35%)	66(-4.35%)	66(-4.35%)

¹ Delta is the generation time's ratio of change compared to the ATLAS dataset.

$$\Delta = (Time_i - Time_{ATLAS}) \times 100\% / Time_{ATLAS}$$

Finding 7: The implementations of focal methods bring additional time cost for DLAGs. IR faces huge generation time cost despite its relatively high accuracy and BLEU score.

5 DISCUSSION

In this section, we discuss the limitations of our work, the impacts of focal methods on DLAGs, and recommendations on identifying focal methods for using DLAGs.

5.1 Limitations

Our methodology has the limitation that our empirical evaluation of DLAGs does not use the ground truth of practical focal methods but is still an approximation of practical focal methods. We have made every effort to ensure that the focal methods in our updated benchmarks are as close as possible to those used in practice.

Another limitation of our work is that we focus on only focal methods in the ATLAS dataset; however, the ATLAS dataset still suffers from other shortcomings. For example, the ATLAS dataset keeps test cases with only a single assertion, but in practical scenarios, one unit test case can contain multiple assertions. Future work can consider improving other aspects of the ATLAS dataset toward realistic usage of DLAGs.

In addition, as the reasons mentioned in Section 3.1, some of the test-to-code traceability techniques, which may be better at identifying focal methods than the selected techniques, are discarded in our experiments. Although making a great effort to include some of the unselected traceability techniques is possible and can improve the richness of the experiments, we hold the view that including more techniques in the experiments does not affect the results and conclusions of the experiments, because the decline in models' capability of generating assertions is independent of the traceability techniques used in the construction of the benchmarks in our experiments.

5.2 How Focal Method Identification Impacts DLAGs

Our experimental results indicate that the strategy of focal method identification in training sets impacts the accuracy of assertion generation for ATLAS and T5, with minimal effects observed for IR and TOGA. One possible explanation is that focal methods in training sets can change the convergence point of the models, potentially leading to an improvement in the capability of assertion generation. Another plausible explanation is that focal methods in expected assertions can be hints for assertion generation, e.g., hints of the observer methods invoked in assertions, and neural network language models can learn from these hints. For IR, test prefixes of high-text-similarity *focal-tests* also exhibit textual similarity, leading to no significant difference in accuracy between using *focal-tests* and using only test prefixes. For TOGA, the return value of a focal

method is claimed to be used to distinguish assertion types within the candidate set. However, the assertion type is determined by parameters in the expected assertion and unrelated to the focal method implementation, which can be even absent in many cases. In other words, the existing implementation of IR and TOGA underutilizes the information in focal methods. This finding highlights a potential avenue for future research and improvement. We believe that this finding challenges the existing understanding of the extent to which focal methods influence assertion generation, bringing a new perspective on focal methods to the research community.

5.3 Recommendations on Identifying Focal Methods for DLAGs

In this section, we present some practical recommendations for using DLAGs. A crucial factor in maximizing a model's capability for assertion generation is ensuring consistency in focal method identification between the training and inference phases, a consideration overlooked in previous research work (training with LCBA but inferring with developer-intended focal methods). For the ATLAS approach, we suggest using the combined-score technique to identify focal methods during both the training and inference phases. Additionally, we recommend using our TS_{atlas} to train T5 models for assertion generation. However, for IR and TOGA, it is hard to decide the best identification technique. We strongly advise developers to either skip or simplify the focal method identification steps and use only test prefixes as input when using IR and TOGA for assertion generation, given that the existing implementations of IR and TOGA do not fully leverage the information from focal methods. Furthermore, considering that including focal method implementations in the training set does not significantly enhance the models' capabilities, as evidenced by comparing results between the original ATLAS dataset and TS_{atlas} (the sole difference being the inclusion of focal method implementation), and also incurs additional time costs, we recommend developers to remove the focal method implementations when using DLAGs.

6 THREATS TO VALIDITY

A potential threat to the validity of our empirical study is the metrics that we use. We use the accuracy metric to evaluate the capability of DLAGs to accurately generate assertions. The accuracy metric is the proportion of perfect prediction, i.e., the generated assertion is the same as the ground truth. However, there are instances where equivalent assertions have different forms, such as assertions with arguments in a different order. These equivalent assertions should also be considered correct because their effects are identical. We do not introduce a metric to reflect the proportion of equivalent assertions because our analysis of incorrectly predicted assertions reveals that the proportion of equivalent assertions is small.

There are two reasons that we do not use the compilable and bug-finding metrics in our experiments. First, the metrics used in our study are sufficient to measure the effectiveness of DLAGs when facing different focal methods. Second, it is challenging to automatically compile large-scale projects because building projects highly depends on external/internal settings/resources [10, 16]. We notice that the bug-finding metric has been used to evaluate TOGA [6, 15] on the Defects4J benchmark [12], and we plan to use this evaluation metric to evaluate all DLAGs in future work.

7 CONCLUSION

In this paper, we have pointed out two issues of the existing empirical evaluation of deep-learning-based approaches for assertion generation (DLAGs), causing inaccurate assessment of DLAGs' suitability for practical use. We have conducted a study of seven test-to-code traceability techniques in the scenario of assertion generation. We have found that the LCBA technique, commonly used in existing DLAGs, is not the best among the seven techniques and can accurately identify focal methods with only 43.38% precision and 38.42% recall; thus, the LCBA technique cannot accurately

reflect developer-intended focal methods, raising a concern about using the ATLAS dataset for evaluation. To address these issues and evaluate DLAGs with practical focal methods, we have constructed a new dataset named ATLAS+ with updated focal methods. We have found that all of the four DLAGs under study achieve lower accuracy on a test set from ATLAS+ than the corresponding test set from the ATLAS dataset, indicating that DLAGs should be (re)evaluated with a test set from ATLAS+, which better reflects practical scenarios of providing focal methods than the ATLAS dataset. We also have found that using a training set in ATLAS+ helps effectively improve the accuracy of the ATLAS approach and T5 approach over these approaches trained using the corresponding training set from the ATLAS dataset. To ensure DLAGs' effectiveness, we recommend that, when using DLAGs in practice, developers should identify focal methods in the same way as those in the corresponding training set, rather than using different identification techniques for training and inference as in previous work [6, 18, 39, 43].

ACKNOWLEDGMENTS

This work was partially supported by the National Natural Science Foundation of China under Grant No. 62161146003, and the Tencent Foundation/XPLORER PRIZE. We would like to thank Zongyang Li, Tianyu Chen, and Weibo Xu for their valuable support for this work. Tao Xie is also affiliated with the Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education China, and is the corresponding author.

REFERENCES

- [1] 2024. https://github.com/Yibo-He/Artifact-ATLAS_Plus-FSE24.
- [2] Miltiadis Allamanis. 2019. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. 143–153. <https://doi.org/10.1145/3359591.3359735>
- [3] M. Moein Almasi, Hadi Hemmati, Gordon Fraser, Andrea Arcuri, and Jundefinednis Benefelds. 2017. An Industrial Evaluation of Unit Test Generation: Finding Real Faults in a Financial Application. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice Track*. 263–272. <https://doi.org/10.1109/ICSE-SEIP.2017.27>
- [4] Pietro Braione, Giovanni Denaro, and Mauro Pezzè. 2016. JBSE: A Symbolic Executor for Java Programs with Complex Heap Inputs. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 1018–1022. <https://doi.org/10.1145/2950290.2983940>
- [5] Ermira Daka and Gordon Fraser. 2014. A Survey on Unit Testing Practices and Problems. In *Proceedings of the 25th IEEE International Symposium on Software Reliability Engineering*. 201–211. <https://doi.org/10.1109/ISSRE.2014.11>
- [6] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. 2022. TOGA: A Neural Method for Test Oracle Generation. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering*. 2130–2141. <https://doi.org/10.1145/3510003.3510141>
- [7] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic Test Suite Generation for Object-Oriented Software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. 416–419. <https://doi.org/10.1145/2025113.2025179>
- [8] Salton Gerard and Michael J. McGill. 1983. *Introduction to Modern Information Retrieval*. McGraw-Hill College.
- [9] Mohammad Ghafari, Carlo Ghezzi, and Konstantin Rubinov. 2015. Automatically Identifying Focal Methods Under Test in Unit Test Cases. In *Proceedings of the 15th IEEE International Working Conference on Source Code Analysis and Manipulation*. 61–70. <https://doi.org/10.1109/SCAM.2015.7335402>
- [10] Foyzul Hassan and Xiaoyin Wang. 2018. HireBuild: An Automatic Approach to History-Driven Repair of Build Scripts. In *Proceedings of the 40th IEEE/ACM International Conference on Software Engineering*. 1078–1089. <https://doi.org/10.1145/3180155.3180181>
- [11] Tianxing He, Shengcheng Yu, Ziyuan Wang, Jieqiong Li, and Zhenyu Chen. 2019. From Data Quality to Model Quality: An Exploratory Study on Deep Learning. In *Proceedings of the 11th Asia-Pacific Symposium on Internetwork*. 1–6. <https://doi.org/10.1145/3361242.3361260>
- [12] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *Proceedings of the 23rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 437–440. <https://doi.org/10.1145/2610384.2628055>

- [13] Manabu Kamimura and Gail C. Murphy. 2013. Towards Generating Human-Oriented Summaries of Unit Test Cases. In *Proceedings of the 21st IEEE International Conference on Program Comprehension*. 215–218. <https://doi.org/10.1109/ICPC.2013.6613851>
- [14] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 7871–7880. <https://doi.org/10.18653/v1/2020.acl-main.703>
- [15] Zhongxin Liu, Kui Liu, Xin Xia, and Xiaohu Yang. 2023. Towards More Realistic Evaluation for Neural Test Oracle Generation. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 589–600. <https://doi.org/10.1145/3597926.3598080>
- [16] Yiling Lou, Zhenpeng Chen, Yanbin Cao, Dan Hao, and Lu Zhang. 2020. Understanding Build Issue Resolution in Practice: Symptoms and Fix Patterns. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 617–628. <https://doi.org/10.1145/3368089.3409760>
- [17] Antonio Mastropaolo, Nathan Cooper, David Nader Palacio, Simone Scalabrino, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2023. Using Transfer Learning for Code-Related Tasks. *IEEE Transactions on Software Engineering* 49, 4 (Apr 2023), 1580–1598. <https://doi.org/10.1109/TSE.2022.3183297>
- [18] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the Usage of Text-To-Text Transfer Transformer to Support Code-Related Tasks. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*. 336–347. <https://doi.org/10.1109/ICSE43902.2021.00041>
- [19] Microsoft. 2022. TOGA. <https://github.com/microsoft/toga>.
- [20] Aleksandar Milicevic, Sasa Misailovic, Darko Marinov, and Sarfraz Khurshid. 2007. Korat: A Tool for Generating Structurally Complex Test Inputs. In *Proceedings of the 29th IEEE/ACM International Conference on Software Engineering*. 771–774. <https://doi.org/10.1109/ICSE.2007.48>
- [21] Moses. 2017. Multi-BLEU. <https://github.com/moses-smt/mosesdecoder/blob/master/scripts/generic/multi-bleu.perl>. Retrieved on March 28, 2023.
- [22] NVIDIA NGC. 2017. TensorFlow Container. <https://catalog.ngc.nvidia.com/orgs/nvidia/containers/tensorflow>. Retrieved on March 28, 2023.
- [23] OpenAI. 2022. ChatGPT. <https://chat.openai.com>.
- [24] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: Feedback-Directed Random Testing for Java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion*. 815–816. <https://doi.org/10.1145/1297846.1297902>
- [25] Reza Meimandi Parizi, Sai Peck Lee, and Mohammad Dabbagh. 2014. Achievements and Challenges in State-of-the-Art Software Traceability Between Test and Code Artifacts. *IEEE Transactions on Reliability* 63, 4 (Dec 2014), 913–926. <https://doi.org/10.1109/TR.2014.2338254>
- [26] Abdallah Qusef, Gabriele Bavota, Rocco Oliveto, Andrea De Lucia, and David Binkley. 2011. SCOTCH: Test-to-code Traceability using Slicing and Conceptual Coupling. In *Proceedings of the 27th IEEE International Conference on Software Maintenance*. 63–72. <https://doi.org/10.1109/ICSM.2011.6080773>
- [27] Abdallah Qusef, Rocco Oliveto, and Andrea De Lucia. 2010. Recovering Traceability Links between Unit Tests and Classes under Test: An Improved Method. In *Proceedings of the 26th IEEE International Conference on Software Maintenance*. 1–10. <https://doi.org/10.1109/ICSM.2010.5609581>
- [28] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 1 (Jan 2020). <https://doi.org/10.5555/3455716.3455856>
- [29] Bart Van Rompaey and Serge Demeyer. 2009. Establishing Traceability Links between Unit Test Cases and Units under Test. In *Proceedings of the 7th European Conference on Software Maintenance and Reengineering*. 209–218. <https://doi.org/10.1109/CSMR.2009.39>
- [30] Sina Shamshiri. 2015. Automated Unit Test Generation for Evolving Software. In *Proceedings of the 10th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1038–1041. <https://doi.org/10.1145/2786805.2803196>
- [31] Weifeng Sun, Meng Yan, Zhongxin Liu, Xin Xia, Yan Lei, and David Lo. 2023. Revisiting the Identification of the Co-evolution of Production and Test Code. *ACM Trans. Softw. Eng. Methodol.* 32, 6 (Sep 2023). <https://doi.org/10.1145/3607183>
- [32] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to Sequence Learning with Neural Networks. In *Proceedings of the 27th International Conference on Neural Information Processing Systems*. 3104–3112. <https://doi.org/10.48550/arXiv.1409.3215>
- [33] T. T. Tanimoto. 1958. *An Elementary Mathematical Theory of Classification and Prediction*. IBM Internal Report.

- [34] Chris Thunes. 2019. Javalang. <https://github.com/c2nes/javalang>. Retrieved on March 28, 2023.
- [35] Michele Tufano, Shao Kun Deng, Neel Sundaresan, and Alexey Svyatkovskiy. 2022. Methods2Test: A Dataset of Focal Methods Mapped to Test Cases. In *Proceedings of the 19th IEEE/ACM International Conference on Mining Software Repositories*. 299–303. <https://doi.org/10.1145/3524842.3528009>
- [36] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. 2022. Generating Accurate Assert Statements for Unit Test Cases using Pretrained Transformers. In *Proceedings of the 17th IEEE/ACM International Conference on Automation of Software Test*. 54–64. <https://doi.org/10.1145/3524481.3527220>
- [37] Sinan Wang, Ming Wen, Yepang Liu, Ying Wang, and Rongxin Wu. 2021. Understanding and Facilitating the Co-Evolution of Production and Test Code. In *Proceedings of the 28th IEEE International Conference on Software Analysis, Evolution and Reengineering*. 272–283. <https://doi.org/10.1109/SANER50967.2021.00033>
- [38] Cody Watson. 2019. ATLAS Appendix. <https://sites.google.com/view/atlas-nmt/home>. Retrieved on March 28, 2023.
- [39] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On Learning Meaningful Assert Statements for Unit Test Cases. In *Proceedings of the 42nd IEEE/ACM International Conference on Software Engineering*. 1398–1409. <https://doi.org/10.1145/3377811.3380429>
- [40] Robert White and Jens Krinke. 2022. TCTracer: Establishing Test-to-Code Traceability Links Using Dynamic and Static Techniques. *Empirical Software Engineering* 27, 3 (May 2022). <https://doi.org/10.1007/s10664-021-10079-1>
- [41] Robert White, Jens Krinke, and Raymond Tan. 2020. Establishing Multilevel Test-to-Code Traceability Links. In *Proceedings of the 42nd IEEE/ACM International Conference on Software Engineering*. 861–872. <https://doi.org/10.1145/3377811.3380921>
- [42] Tao Xie. 2006. Augmenting Automatically Generated Unit-Test Suites with Regression Oracle Checking. In *Proceedings of the 20th European Conference on Object-Oriented Programming*. 380–403. https://doi.org/10.1007/11785477_23
- [43] Hao Yu, Yiling Lou, Ke Sun, Dezhi Ran, Tao Xie, Dan Hao, Ying Li, Ge Li, and Qianxiang Wang. 2022. Automated Assertion Generation via Information Retrieval and Its Integration with Deep Learning. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering*. 163–174. <https://doi.org/10.1145/3510003.3510149>
- [44] Benwen Zhang, Emily Hill, and James Clause. 2016. Towards Automatically Generating Descriptive Names for Unit Tests. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*. 625–636. <https://doi.org/10.1145/2970276.2970342>

Received 2023-09-28; accepted 2024-04-16